

Listas em Python (Vetor)

Prof. Laercio Brito Gonçalves

Listas (Vetor)

- Exemplo1:

Faça um programa que leia matricula e nota de 5 alunos e em seguida imprima as matriculas dos alunos cuja nota seja maior ou igual a média das 5 notas.

Listas (Vetor)

```
nome1=input('Entre com o nome1: ')
nota1=float(input('Entre com a nota1: '))
nome2=input('Entre com o nome2: ')
nota2=float(input('Entre com a nota2: '))
nome3=input('Entre com o nome3: ')
nota3=float(input('Entre com a nota3: '))
nome4=input('Entre com o nome4: ')
nota4=float(input('Entre com a nota4: '))
nome5=input('Entre com o nome5: ')
nota5=float(input('Entre com a nota5: '))
```

```
media = (nota1 + nota2 + nota3 + nota4 + nota5)/5
```

```
if (nota1>media):
    print(f'{nome1} possui nota maior que a média ')
if (nota2>media):
    print(f'{nome2} possui nota maior que a média ')
if (nota3>media):
    print(f'{nome3} possui nota maior que a média ')
if (nota4>media):
    print(f'{nome4} possui nota maior que a média ')
if (nota5>media):
    print(f'{nome5} possui nota maior que a média ')
```

Listas (Vetor)

- E se fossem 100 alunos?
 - É possível definir variáveis que guardam mais de um valor de um mesmo tipo
 - Essas variáveis são conhecidas como variáveis indexáveis ou arranjos (*array*)
 - Em Python essas variáveis são chamadas de ***listas***.

Vetor

- Variável composta **unidimensional**
 - Contém espaço para armazenar diversos valores
 - É acessada via um índice
- A ideia de vetor é comum na matemática, com o nome de variável subscrita

Exemplo: $x_1, x_2, \dots, x_n$

Vetores

- O que vimos até agora são variáveis com somente um valor
 - Exemplo: $y = 123$
- No caso de vetores, uma mesma variável guarda ao mesmo tempo múltiplos valores
 - Exemplo: $x_1 = 123$, $x_2 = 456$, ...
 - $x = [123, 456, \dots]$

Listas

- Em outras linguagens de programação, listas são chamadas de vetores e possuem restrições que Python não tem:
 - Em Python, os valores de uma lista podem ser de qualquer tipo
 - Em outras linguagens, os valores precisam ser do mesmo tipo
- Em Python
 - conjunto = ['A', 1, 2, 'Casa', 2.3]
 - notas = [10, 5, 6.7, 2, 7.5]

Listas

- Para acessar (ler ou escrever) uma posição do vetor, basta informar a posição entre colchetes

```
notas = [8.0, 5.5, 1.5]  
media = (notas[0] + notas[1] + notas[2]) / 3
```

notas	0	8.0
	1	5.5
	2	1.5
media		5.0

Listas

- Pode-se iterar por todos os seus valores usando um comando **for**

```
notas = [8.0, 5.5, 1.5]
for i in range(3):
    print(notas[i])
```

Como Inserir Elementos na Lista?

- Inicialmente cria-se uma lista vazia
 - Ex.: `notas = []`
- Depois adiciona-se valores na lista usando **append**
 - Ex.:

```
n = float(input('Digite a nota do primeiro aluno: '))
notas.append(n)
```

Listas

- Exemplo2:

Faça um programa que leia matricula e nota de 100 alunos e em seguida imprima as matriculas dos alunos cuja nota seja maior ou igual a média das 100 notas.

Listas

- Exemplo2:

Faça um programa que leia matrícula e nota de 100 alunos e em seguida imprima as matrículas dos alunos cuja nota seja maior ou igual a média das 100 notas.

Será necessário criar duas listas (vetores) com 100 posições em cada lista.

- Uma lista para nomes
- Uma lista para notas

Listas

```
tot_alu = 100
```

```
nota = []
```

```
nome = []
```

```
media = 0
```

```
for i in range(tot_alu):
```

```
    nome.append(input(f'Entre com o nome[{i}]: '))
```

```
    nota.append(float(input(f'Entre com a nota[{i}]: ')))
```

```
    media += nota[i]
```

```
media/=3
```

```
print(f'Alunos que possuem nota maior que a média das {tot_alu} notas:')
```

```
for i in range(tot_alu):
```

```
    if (nota[i]>=media):
```

```
        print(nome[i])
```

Índices para Acesso aos Elementos da Lista

- Python permite acesso à lista em ordem crescente ou decrescente de posição
 - Primeira posição é 0
 - Última posição é -1

```
>>> c = [-45, 6, 0, 72, 1543]
>>> c[3]
72
>>> c[-2]
72
>>> c[0] = c[-5]
True
```

c[0]	-45	c[-5]
c[1]	6	c[-4]
c[2]	0	c[-3]
c[3]	72	c[-2]
c[4]	1543	c[-1]

Função len

- Retorna a quantidade de elementos que a lista possui

```
>>> numeros = [3,1,6,7,10,22,4]
>>> len(numeros)
7
```

Concatenação de Listas

- É possível anexar os valores de uma lista em outra usando o operador “+”

```
>>> lista = [1,2,3]
>>> lista = lista + [4]
[1,2,3,4]
>>> lista = lista + [4,5,6]
[1,2,3,4,4,5,6]
```

Concatenação de Listas

- Exemplo3:

Faça um programa que leia um número inteiro n , e retorne uma lista com todos os números pares entre 2 e um número n , inclusive.

Concatenação de Listas

- Exemplo3:

Faça um programa que leia um número inteiro n, e retorne uma lista com todos os números pares entre 2 e um número n, inclusive.

```
lista = []
```

```
n=int(input('Entre com um número inteiro:'))
```

```
for i in range(2,n+1,2):  
    lista += [i];
```

```
print(lista)
```

Concatenação de Listas

- Exemplo3:

Faça um programa que leia um número inteiro n, e retorne uma lista com todos os números pares entre 2 e um número n, inclusive.

```
lista = []
```

```
Entre com um número inteiro:10  
[2, 4, 6, 8, 10]
```

```
n=int(input('Entre com um número inteiro:'))
```

```
for i in range(2,n+1,2):  
    lista += [i];
```

```
print(lista)
```

Concatenação de Listas

- Exemplo4:

Faça um programa que leia um número inteiro n , e retorne uma lista com todos os números pares entre 2 e um número n , inclusive, em ordem reversa.

Concatenação de Listas

- Exemplo4:

Faça um programa que leia um número inteiro n, e retorne uma lista com todos os números pares entre 2 e um número n, inclusive, em ordem reversa.

```
lista = []

n=int(input('Entre com um número inteiro:'))

for i in range(2,n+1,2):
    lista = [i] + lista;

print(lista)
```

Concatenação de Listas

- Exemplo4:

Faça um programa que leia um número inteiro n, e retorne uma lista com todos os números pares entre 2 e um número n, inclusive, em ordem reversa.

```
lista = []
```

```
n=int(input('Entre com um número inteiro:'))
```

```
for i in range(2,n+1,2):  
    lista = [i] + lista;
```

```
print(lista)
```

Entre com um número inteiro:10
[10, 8, 6, 4, 2]

“Multiplicação” de Listas

- O operador “*” repete **n** vezes os elementos que já estão na lista
- A operação **lista * n** equivale a **lista + lista + ... + lista** (n vezes)

```
>>> lista = [1,2,3]
>>> lista = lista * 3
[1,2,3,1,2,3,1,2,3]
```

Inicialização de Listas com Zero

- Em diversas situações onde **já sabemos de antemão qual será o tamanho de uma lista** de inteiros, é útil inicializar a lista com o valor 0
- Isso evita que precisemos usar o **append** para adicionar valores

```
>>> tamanho = 10
>>> lista = [0] * tamanho
>>> lista
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Inicialização de Listas com Zero

- Exemplo5:

Faça um programa que crie uma lista com 5 notas e em seguida calcule a média, **sem utilizar append**

Inicialização de Listas com Zero

- Exemplo5:

Faça um programa que crie uma lista com 5 notas e em seguida calcule a média, **sem utilizar append**

```
tam = 5
lista = [0]*tam
media = 0

for i in range(tam):
    lista[i] = int(input(f'Entre com lista[{i}]: '))
    media += lista[i]

media/=tam
print(lista)
print('Média = ',media)
```

Teste de Pertinência

Teste de Pertinência – Elemento `in` Lista

- Elemento `in` lista
- Verifica se um determinada valor pertence ou não pertence a uma lista
- Retornar True caso o valor pertença à lista, e False caso contrário

Teste de Pertinência – Elemento **in** Lista

- Exemplo6:

Verificar se o número 7 pertence a lista abaixo.

Teste de Pertinência – Elemento `in` Lista

- Exemplo6:

Verificar se o número 7 pertence a lista abaixo.

```
lista = [1,2,3,4,5]  
resultado = 7 in lista  
print(resultado)
```

Teste de Pertinência – Elemento `in` Lista

- Exemplo6:

Verificar se o número 7 pertence a lista abaixo.

```
lista = [1,2,3,4,5]  
resultado = 7 in lista  
print(resultado)
```

False

Teste de Pertinência – Elemento **in** Lista

- Exemplo7:

Verificar se o número 7 pertence a lista abaixo, sem usar **in**.

Teste de Pertinência – Elemento **in** Lista

- Exemplo7:

Verificar se o número 7 pertence a lista abaixo, sem usar **in**.

```
lista = [1,2,3,4,5]
valor = 7
resultado = False

for i in range(len(lista)):
    if lista[i]==valor:
        resultado = True

print(resultado)
```

Teste de Pertinência – Elemento `in` Lista

- Exemplo7:

Verificar se o número 7 pertence a lista abaixo, sem usar `in`.

```
lista = [1,2,3,4,5]
valor = 7
resultado = False

for i in range(len(lista)):
    if lista[i]==valor:
        resultado = True

print(resultado)
```

False

Conhecimento útil: **split()** retorna um vetor

- Utilizado na função `input()`
- Só permite criar uma lista de cadeia de caracteres!!!!
- A função `int()` não pode ser utilizada para converter uma lista de cadeia de caracteres em uma lista de inteiros!!!!!!!!!!
- Ex.:

```
lista = input('Entre com valores separados por espaços: ').split()  
print(lista)
```

Conhecimento útil: **split()** retorna um vetor

- Utilizado na função `input()`
- Só permite criar uma lista de cadeia de caracteres!!!!
- A função `int()` não pode ser utilizada para converter uma lista de cadeia de caracteres em uma lista de inteiros!!!!!!!!!!
- Ex.:

```
lista = input('Entre com valores separados por espaços: ').split()  
print(lista)
```

```
Entre com valores separados por espaços: 1 2 3 4 5 6  
['1', '2', '3', '4', '5', '6']
```

Representação de Listas em Memória

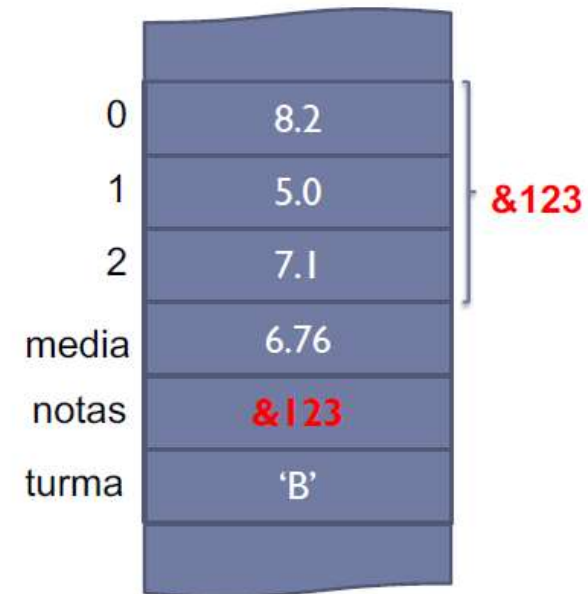
Representação de Listas em Memória

- O valor de uma variável de lista na verdade é um endereço de memória

Em Python

```
notas = [8.2, 5.0, 7.1]
turma = 'B'
media = 0
for i in range(len(notas)):
    media = media + notas[i]
media = media/len(notas)
```

Na Memória



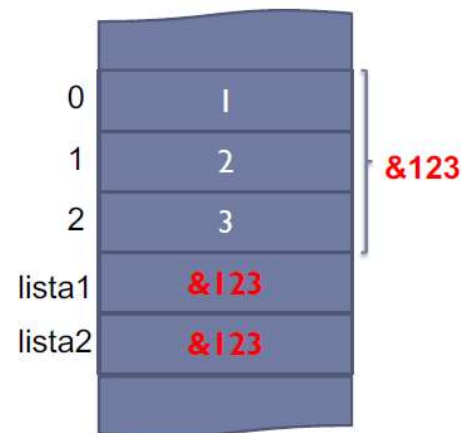
Cópia de Listas

- Ao **copiar uma lista para outra**, o que é feito é **copiar** o valor do endereço de memória
- **Ambas** passam a apontar para o **mesmo endereço**, portanto o que for modificado em uma lista também será modificado na outra

Em Python

```
>>> lista1 = [1, 2, 3]
>>> lista2 = lista1
```

Na Memória

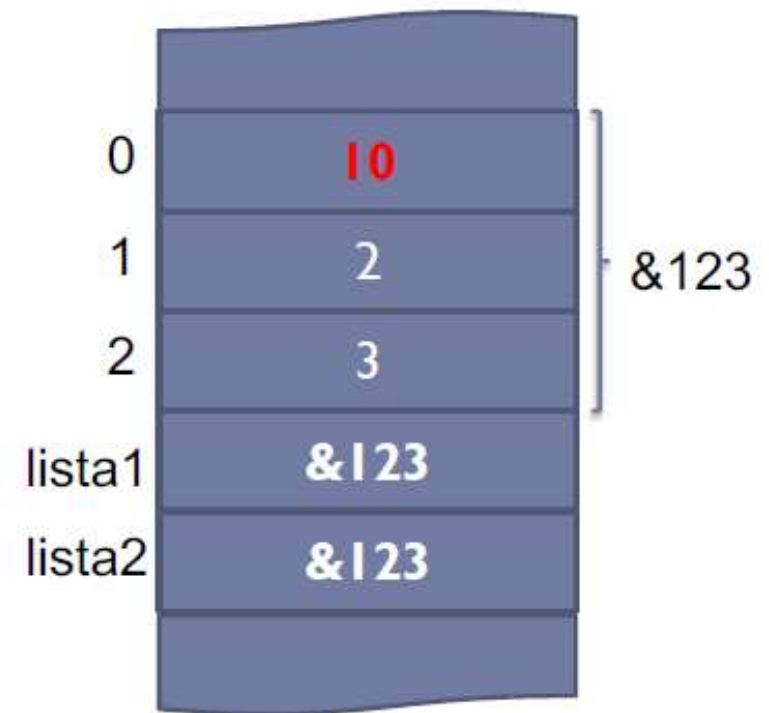


Cópia de Listas

Em Python

```
>>> lista1 = [1, 2, 3]
>>> lista2 = lista1
>>> lista1[0] = 10
>>> lista1
[10, 2, 3]
>>> lista2
[10, 2, 3]
```

Na Memória

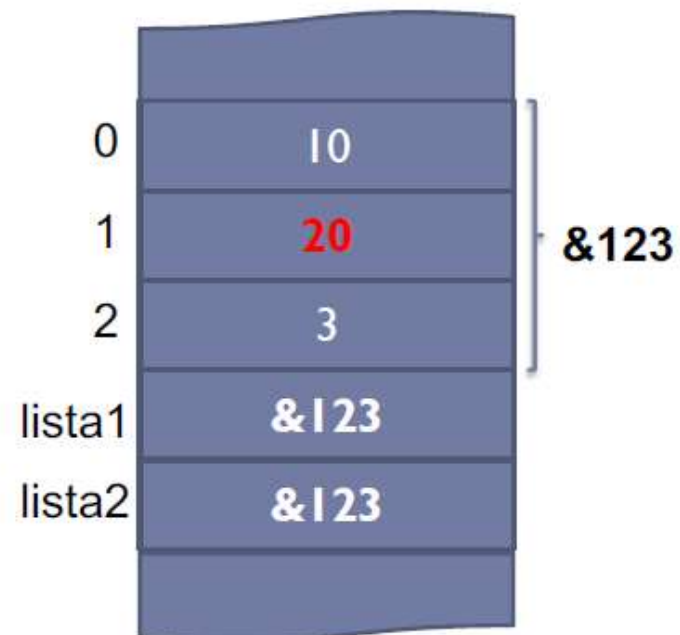


Cópia de Listas

Em Python

```
>>> lista1 = [1, 2, 3]
>>> lista2 = lista1
>>> lista1[0] = 10
>>> lista1
[10, 2, 3]
>>> lista2
[10, 2, 3]
>>> lista2[1] = 20
>>> lista2
[10, 20, 3]
>>> lista1
[10, 20, 3]
```

Na Memória



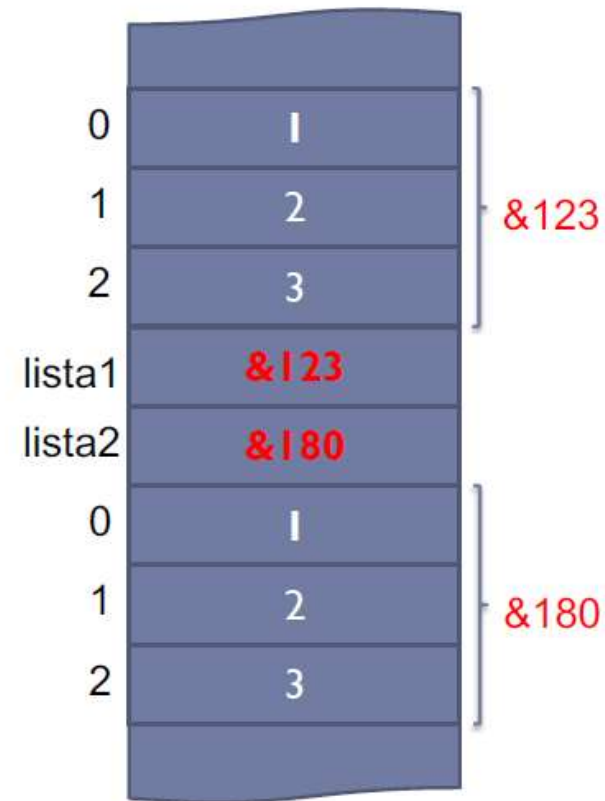
Cópia de Listas

- Como evitar isso?
 - Usar um for para copiar valor por valor

Cópia de Listas

Exemplo

```
>>> lista1 = [1, 2, 3]
>>> lista2 = []
>>> for i in range(len(lista1)):
...     lista2.append(lista1[i])
... 
```



Cópia de Listas

Dessa forma:

- ▶ Alterações em uma lista não são refletidas na outra

```
>>> lista2 = []
```

```
>>> lista1 = [1, 2, 3, 4, 5]
```

```
>>> for i in range(len(lista1)):
```

```
...     lista2.append(lista1[i])
```

```
>>> lista2[0] = 10
```

```
>>> lista1
```

```
[1, 2, 3, 4, 5]
```

```
>>> lista2
```

```
[10, 2, 3, 4, 5]
```

```
>>> lista1[3] = 20
```

```
>>> lista2
```

```
[10, 2, 3, 4, 5]
```

Cópia de Listas – Outra maneira

Ex.:

```
lista1=[1,2,3,4,5,6]
```

```
lista2=lista1
```

```
lista3=lista1[:]
```

```
print(id(lista1))
```

```
print(id(lista2))
```

```
print(id(lista3))
```

Resposta

1912293437248

1912293437248

1912293435072

Cópia de Listas – Outra maneira

Ex.:

```
lista1=[1,2,3,4,5,6]  
lista2=lista1  
lista3=lista1[:]
```

```
print(id(lista1))  
print(id(lista2))  
print(id(lista3))
```

Resposta

```
1912293437248  
1912293437248  
1912293435072
```

lista1 e lista2 possuem o mesmo endereço

lista3 tem outro endereço de memória

Como apagar um elemento de uma Lista?

- Função **del()**

```
>>> lista = ['a','b','c','d','e','f']
```

```
>>> lista
```

```
['a','b','c','d','e','f']
```

```
>>> del(lista[2])
```

```
>>> lista
```

```
['a','b','d','e','f']
```

```
>>> lista = ['a','b','c','d','e','f']
```

```
>>> del(lista[1:5])
```

```
>>> lista
```

```
['a', 'f']
```

Como contar o número de ocorrências em uma Lista?

- **count()**

Exemplo:

```
lista = [1,2,1,2,1,2,3,4,5]  
print(lista.count(1))
```

Resposta:

3

Menor e Maior Elemento de uma Lista

- `min()` → retorna o menor elemento da lista
- `max()` → retorna o maior elemento da lista

Ex.:

```
lista = [1,2,3,4,5,6]
```

```
print(min(lista))
```

```
print(max(lista))
```

Resposta:

1

6

Retorna o índice de um elemento no vetor

- Pode-se encontrar a posição de um elemento em um vetor utilizando o método ***index()***.
- Exemplo:

```
vetor = [5, 3, 10, 1, 98]  
print(vetor.index(10))
```

Retorna o índice de um elemento no vetor

- Pode-se encontrar a posição de um elemento em um vetor utilizando o método ***index()***.

- Exemplo:

```
vetor = [5, 3, 10, 1, 98]  
print(vetor.index(10))
```

10 está na posição 2 do vetor
(a contagem começa da posição 0)

Resposta:
2

Retorna o índice de um elemento no vetor

- Se houver elementos repetidos no vetor, o método ***index()*** irá retornar a posição do primeiro valor encontrado

- Exemplo:

```
vetor = [5, 3, 10, 10, 98]  
print(vetor.index(10))
```

10 está na posição 2 do vetor
(a contagem começa da posição 0)

Resposta:
2

Função sorted()

- Ordena uma lista

```
a = [1, 2, -7, 89, 43, 14]
```

```
b = sorted(a)
```

```
print(a)
```

```
print(b)
```

```
[1, 2, -7, 89, 43, 14]
```

```
[-7, 1, 2, 14, 43, 89]
```

Exercícios:

1. Faça um programa que leia dois vetores de 3 posições, que representam forças sobre um ponto no espaço 3D, e escreva a força resultante. Dica: força resultante é obtida pela soma dos valores das coordenadas correspondentes nos dois vetores: $(x_1 + x_2)$, $(y_1 + y_2)$, $(z_1 + z_2)$
2. Faça um programa que preencha por leitura um vetor de 10 posições, e conta quantos valores diferentes existem no vetor.
3. Faça um programa que preencha por leitura um vetor de 5 posições, e informe a posição em que um valor x (lido do teclado) está no vetor. Caso o valor x não seja encontrado, o programa deve imprimir o valor -1. Faça a questão de duas formas: não utilizando o método `index()` e utilizando o método `index()`.

Exercícios:

4. Um dado é lançado 50 vezes, e o valor correspondente é armazenado em um vetor. Faça um programa que determine o percentual de ocorrências de face 6 do dado dentre esses 50 lançamentos. Obs: utilize a função `randint()` para gerar os 50 lançamentos do dados. `randint(1,6)` → gera números inteiros aleatórios de 1 a 6. Insira no seu programa:

```
from random import randint
```

Exercícios:

5. Faça um programa que leia um vetor **vet** de 20 números inteiros. O programa deve gerar, a partir do vetor lido, um outro vetor **pos** que contenha apenas os valores inteiros positivos de **vet**. A partir do vetor **pos**, deve ser gerado um outro vetor **semdup** que contenha apenas uma ocorrência de cada valor de **pos**.

Exercícios:

6. Dado um vetor de 100 elementos, determine o maior e o menor elemento do vetor. (Utilize as funções `min()` e `max()`)
7. Dado um vetor de 100 elementos, determine o maior e o menor elemento do vetor. (Não utilize as funções `min()` e `max()`)
8. Dado um vetor de 100 elementos, determine a diferença entre a soma dos elementos de índice par e a soma dos elementos de índice ímpar.
9. Dado um vetor com 20 elementos, gerar outro vetor que contenha somente números múltiplos de 3 encontrados no primeiro vetor.
10. Existem 20 candidatos à presidência, com códigos que variam de 1 até 20. Codificou-se 21 para votos brancos e 22 para votos nulos. Cada voto vem em um cartão (contendo só um voto) e o último cartão vem com o número 0 (zero). Faça um programa que auxilie a operação dos votos, imprimindo a quantidade de votos de cada candidato, o número de votos em branco, o número de votos nulos e o total de votantes. Imprima também o(s) candidato(s) que venceram a eleição e o número de votos do(s) vencedor(es).